

Long Multiplication

1000 $\rightarrow$ Multiplicand

$\times$ 1001 $\rightarrow$ Multiplier

$$\begin{array}{r} 0001000 \\ 0000000 \\ 0000000 \\ 1000000 \\ \hline 1001000 \end{array} \rightarrow \text{product}$$

Maximum size of product

$\hookrightarrow$ length of multiplicand

+ " " multiplier

* In the RISC-V command
when we add two numbers,
both have to be same size

* Red 0's are given to point that fact out

* Loop will run for the long multiplication for the amount of bits the multiplier has, in above example it is 4

* $\begin{array}{r} 1000 \\ 1001 \end{array} \rightarrow$ we fix this position for multiplier rather than changing position like we do normally

* We fix position 0 and

right shift the multiplier
1 time in each iteration
to multiply with rest
of the digits of it
gradually, that is how
we always multiply by the
bit in 0th positioned bit
of multiplier rather
changing position like we do

Multiplicand
[0.....0] [actual
MSB multiplicand]
LSB

Shift left



128 bits

128-bit
ALU

Product

64-bit ; 64-bit
MSB LSB

128 bits

Multiplicand
Shift
Right

64-bits

Control test

- * Control decides when to shift multiplicand, multiplier register, when to use ALU and write data in product
- * Since RISC-V has 64-bit registers, we use two registers to keep the 128-bit values and keep track of MSB and LSB.
- * Since we add multiplicand with product during iterations, and maximum product size is $(64 + 64) \rightarrow 128$ -bits, so we expand the multiplicand

to 128-bit as both numbers need to be of same length to add

* ALU performs the addition

* Control test maintains the sequence of operations by giving signal on which clock pulse will perform what

Optimized Multiplier

product $\rightarrow$ 128-bit
MSB
0 0
64-bit
(Product)

LSB
Multiplier

if (iteration $\leq$ multiplier
bit length):

if (multiplier0 == 1):

product_MSB = Multiplicand
+ Product_MSB

product = right shift
product by 1

elif (multiplier0 == 0):

product = right shift
product by 1

Start

Multiplier₀
= 1

1.
Test
Multiplier₀

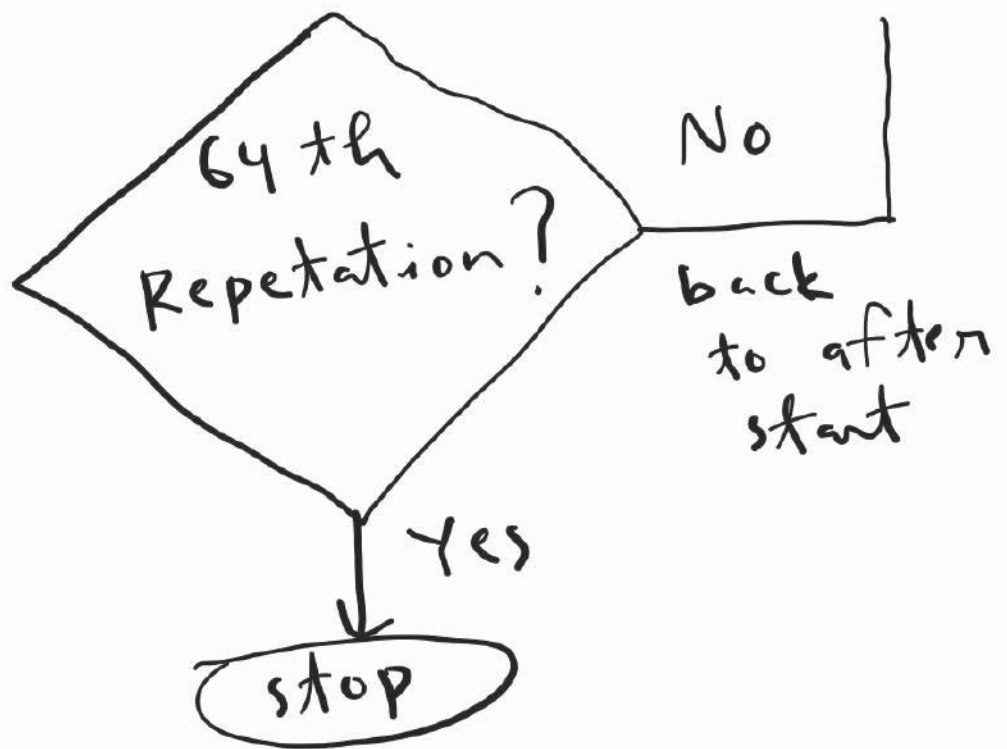
Multiplier₀ = 0

LSB
bit of
multiplier

1a. Add
multiplier to
product and
place result in
product

2. Shift the
multiplier register
left 1-bit

3. Shift the multiplier
register right 1-bit



Now,

$$\begin{array}{r} 1000 \\ \times 1001 \\ \hline \end{array}$$
 → 4-bit so needs 4 iterations

iteration	multiplier	multiplicand	LSB has actual value	product
0	1001	0000 1000		0000 0000
1	1001	0000 1000		0000 1000
	1001	0001 0000		0000 1000
	0100	0001 0000		0000 1000
2				
		0010 0000		0000 1000
	0010	0010 0000		0000 1000

3

0001

0100 0000

0100 0000

0000 1000

0000 1000

4

0000

0100 0000

1000 0000

1000 0000

0100 1000

0100 1000

0100 1000

Loop ended↓
result

IEEE-754 Floating Point Representation

* F_0 to F_{31} stores floating point and real numbers in RISC-V

$$4500000 \rightarrow \underbrace{4.5}_{\text{Co-efficient}} \times \underbrace{10^6}_{\text{base}}$$

exponent
↑
6

* IEEE -754 protocol

defines the rules to represent floating point numbers

* single precision format $\rightarrow$ 32-bit
double " " $\rightarrow$ 64-bit

* A decimal / binary number is normalized if:

i) Only one digit is before the decimal / binary point

ii) And that digit must be a non-zero number

1.23456 $\rightarrow$ normalized

123.456 $\rightarrow$ not normalized

* To normalize a number we need to shift the decimal/binary point to left or right until we get a single non-zero digit before the decimal point

* The shift amount is adjusted by addition or subtraction in the exponent

$$112.54 \times 10^{35}$$

Normalized $\rightarrow 1.1254 \times 10^{37}$

0.0065

Normalized $\rightarrow 6.5 \times 10^{-3}$

$(0.000010101)_2$

$\rightarrow 1.0101 \times 2^{-5}$

IEEE Floating Point Format

	Sign bit	Biased Exponent	Fraction
Single P.	1 bit	8 bits	23 bits
Double P.	1 bit	11 bits	52 bits

32-bit

64-bit

- i) $1. \underbrace{101011}_{6\text{-bit fraction}} \times 2^{39}$
- ii) $1. \underbrace{101011101011}_{12\text{-bit fraction}} \times 2^{-36}$

- * More fraction bits means better accuracy/precision
- * Exponent is always unsigned
- * In example (ii) we have a negative exponent, we convert to exponent to an unsigned number which is called biased exponent and we store the biased exponent

For 8-bit biased exponent,

$$\begin{aligned}\text{Range} &\rightarrow 0 \text{ to } 2^8 - 1 \\ &= 0 \text{ to } 255\end{aligned}$$

But 00000000 and 11111111 are reserved so the range

for biased exponent range is

1 to 254

In double precision,

11-bit biased exponent,

Range $\rightarrow 0$ to $2^{11} - 1$
 $= 0$ to 2047

* 0 and 2047 is reserved,
so the biased exponent
range is 1 to 2046

* Exponent represents how
much big or small of a
number we can represent

For single precision,

$$0 \text{ to } 2^8 - 1$$

$$= 0 \text{ to } 255$$

$$= 1 \text{ to } 254 \rightarrow \text{biased exponent range}$$

Actual exponent range

$$\rightarrow (1 - 127 \text{ to } 254 - 127)$$

$$= -126 \text{ to } +127$$

As Bias in 8-bit system is,

$$2^{8-1} - 1 = 127$$

So,

Highest number represented,

$$\text{~~~~~} \times 2^{127}$$

Lowest number represented,

$$\sim \times 2^{-126}$$

For double precision,

$$0 \text{ to } 2^{11} - 1$$

$$= 0 \text{ to } 2047$$

$$= 1 \text{ to } 2046 \rightarrow \text{biased exponent range}$$

$$\text{Bias} \rightarrow 2^{11-1} - 1 = 1023$$

Actual exponent range

$$\hookrightarrow (1 - 1023 \text{ to } 2046 - 1023)$$

$$= (-1022 \text{ to } +1023)$$

* Bias $\rightarrow 2^{n-1} - 1$, where
n is the size of exponent
field in a floating point
rep system

* Biased Exponent

$\hookrightarrow$ Actual Exponent + Bias

1.1011×2^{34} , find biased
exponent of given number
using single precision format

* First we need to check
whether the number is
in normalized form or
not, if no then we convert
it to normalized form

$$\text{Bias} \rightarrow 2^{8-1} - 1 = 127$$

$n=8$ as single precision

$$\text{Biased exp} = 34 + 127 = 161$$

$$\# 11.1011 \times 2^{-8}, \text{ single precision}$$

$$= 1.11011 \times 2^{-8+1}$$

$$= 1.11011 \times 2^{\textcircled{-7}} \text{ (normalized)}$$

↳ Actual exponent

$$\text{Biased exp} \rightarrow -7 + 127 = \underline{120}$$

unsigned
always

$$\# 11.1011 \times 2^{-8}, \text{ find}$$

biased exponent in 15 bit
IEEE 754 format where
fraction field is 8-bits

Format,

Sign	Biased Exponent	Fraction
------	-----------------	----------

$$1 + x + 8 = 15$$

$\Rightarrow x = 6$, size of exponent

$$\text{Bias} \rightarrow 2^{6-1} - 1 = 31$$

$$11.1011 \times 2^{-8}$$

$$\Rightarrow 1.11011 \times 2^{-7} \text{ (normalized)}$$

$$\text{Biased exponent} \rightarrow -7 + 31 = 24$$

Decimal to Floating Point

Conversion

i) Convert decimal number to binary

- ii) Normalize binary number
- iii) Find biased exponent
- iv) Find sign bit
- v) Find fraction

Convert 50.6749_{10} to
32-bit IEEE 754 format

$$50 \rightarrow 110010$$

$$.6749 \rightarrow 1010110011\dots$$

(Conversion method used in
260) $0.6749 \rightarrow$ Like this

$$\begin{array}{r} 0.6749 \\ \times 2 \\ \hline \end{array}$$

$$50.6749 \rightarrow 110010.1010110011\dots$$

* 10 digit after decimal is
standard

Now,

$$110010.1010110011 \dots \times 2^0$$
$$= 1. \underbrace{100101010110011}_{\text{fraction}} \dots \times 2^5 \quad (\text{normalized done})$$

$$\text{Bias} \rightarrow 2^{8-1} - 1 \quad (\text{single precision})$$
$$= 127$$

$$\text{Biased exponent} \rightarrow 5 + 127 = 132$$

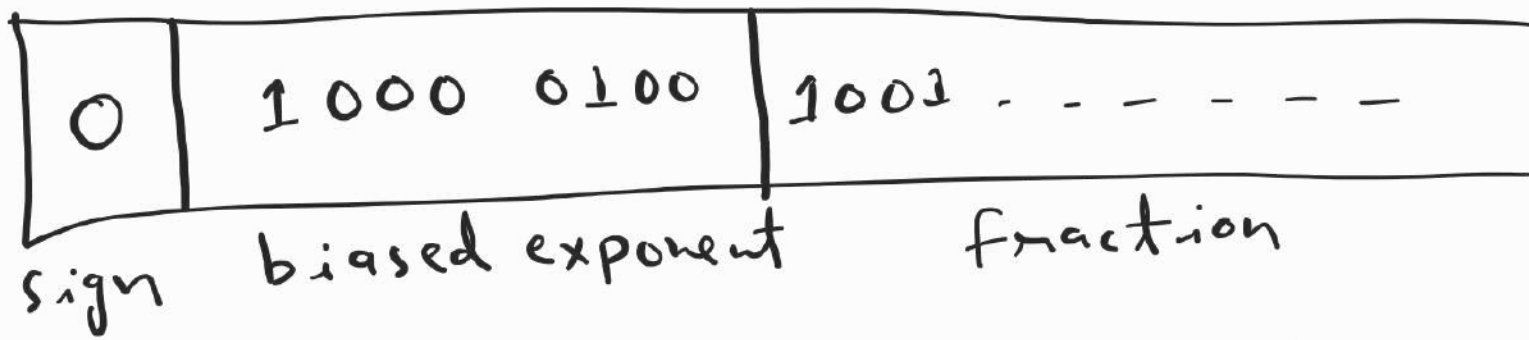
$$132 \rightarrow 10000100$$

$$\text{Sign bit} \rightarrow 0 \quad (\text{since positive})$$

$$\text{Fraction} \rightarrow \begin{array}{ccccccc} 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ & & & & & & 0 & 0 & 0 & 0 & & 0 & 0 & 0 & 0 \end{array}$$

(putting rest of the bits
as 0 since fraction is 23 bit
in single precision format)

Now,



* Convert to hex if asked

0x424AB300 in this case

Convert -0.0232 to 12-bit

IEEE-754 floating point
representation where biased
exponent is 4-bits

Sign $\rightarrow$ 1-bit

fraction $\rightarrow 12 - 1 - 4 = 7$ -bits

$$-0.0232 = -0.0000010\dots$$

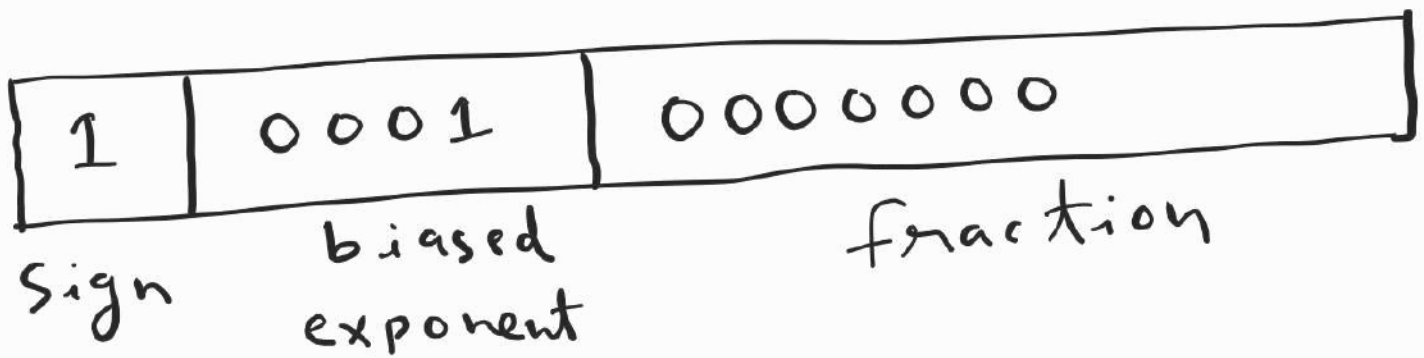
$$= -1.0 \times 2^{-6}$$

(normalized)

$$\text{Bias} \rightarrow 2^{4-1} - 1 = 7$$

$$\text{Biased exponent} \rightarrow -6 + 7 = 1$$

Now,



In hex,

0x880

Floating Point to Decimal

- i) Hex to binary
- ii) Arrange binary according to format
- iii) Determine sign

iv) Find exponent from biased exponent

v) Convert fraction to decimal

vi) Decimal Number

$$\hookrightarrow (-1)^{\text{sign bit}} \times (1 + \text{fraction}) \times 2^{\text{exponent}}$$

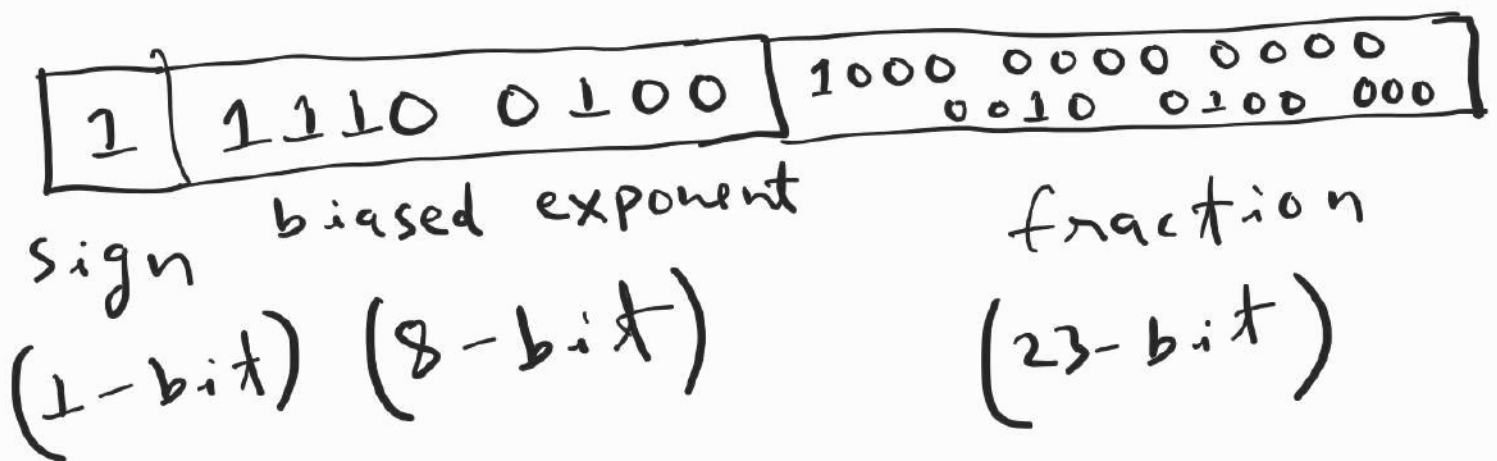
0xF2400120, single precision floating point number to decimal

0xF2400120

Binary,

1111 0010 0100 0000 0000
0001 0010 0000

Format for single precision



Sign bit $\rightarrow 1$

Biased exponent $\rightarrow 1110 0100$
 $\hookrightarrow (228)_{10}$

$$\text{Bias} \rightarrow 2^{n-1} - 1 = 2^{8-1} - 1 = 127$$

$$\text{Actual exponent} \rightarrow 228 - 127 \\ = (101)_{10}$$

$$\text{fraction} = \begin{array}{cccccccc} 0 & . & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ & & 0 & 0 & 0 & 1 & & 0 & 0 & 1 & 0 & & 0 & 0 & 0 & 0 & 0 \end{array}$$

$$= 0.5000343323$$

$$\text{Decimal} = (-1)^1 \times$$

$$(1 + 0.5000343323) \times 2^{101}$$

$$= -1.5000343323 \times 2^{101}$$

$$= -3.80303884 \times 10^{30}$$

Round the answer up to
6 decimal point,

$$-3.803039 \times 10^{30}$$

Up to 6 decimal points without rounding $\rightarrow -3.803038 \times 10^{30}$

Floating Point Addition

- i) Convert both numbers to binary (A and B)
- ii) Normalize A and B
- iii) Align the binary point so that lower exponent matches with higher exponent

- iv) Perform addition
- v) Normalize the result
- vi) Round if asked
- vii) Overflow/Underflow check

$$\# 9.999 \times 10^1 + 1.610 \times 10^{-1}$$

* Firstly we need to
remove the exponents

$$99.99 \times 10^{1-1} \quad \left| \quad 0.1610 \times 10^{-1+1} \right. \\ = 99.99 \quad \left. \quad \quad \quad = 0.1610 \right.$$

So,

$$99.99 + 0.1610$$

$$= 1100011.111111010101$$

$$+ 0.0010100100$$

Normalize,

$$= 1.1000111111110101 \times 2^6$$

$$+ 1.0100100 \times 2^{-3}$$

* Find minimum exponent, which is -3 in this case, change that exponent into the big exponent which is 6 in this case, so $-3+9=6$.
So, 9 place left shift

$$= 1.1000111111110101 \times 2^6$$

$$+ 0.00000000010100100 \times 2^6$$

$$\begin{array}{r}
 1.10000111111110101 \\
 + 0.00000000010100100 \\
 \hline
 1.1001000010011001 \\
 \times 2^6
 \end{array}$$

$$= 1.10010 \times 2^6 \text{ (shortened)}$$

(In normalized form already)

* Question may mention how many points to add after floating point if the number is too long, it may say add up to 8 or 10 points etc.

* Answer will change depending on how many bits we are adding after decimal point

Floating Point Multiplication

- i) Convert to binary
- ii) Normalize both numbers
- iii) Add the exponents
- iv) Perform multiplication
- v) Normalize result
- vi) Round if necessary
- vii) Determine sign from the operation

$$1.110 \times 2^5 \quad \times \quad 111.11 \times 2^{-7}$$

Normalize,

$$1.110 \times 2^5 \quad \times \quad 1.1111 \times 2^{-5}$$

Add exponents since base
is same,

$$(1.110 \times 1.1111) \times 2^{5-5}$$

$$(1.110 \times 1.1111) \times 2^0$$

↓

3 digits

after binary

point

↓
4 digits after
binary point

* So result will have $(4+3) \rightarrow 7$
digits after binary point

In calculator,

$$1110 \times 11111$$

$$\text{result} \rightarrow 110110010$$

7 digits after binary point,

$$\text{So actual result} \rightarrow 11.0110010 \times 2^0$$

Normalize result,

$$1.10110010 \times 2^1$$

* No rounding as not mentioned

* Sign is positive here
since both numbers are +ve

Overflow / Underflow

Let, $\text{range} = (x \text{ to } y)$
lower limit $\swarrow$ $\searrow$ upper limit

a is a biased exponent,

if $(x \leq a \leq y)$:

No overflow/underflow

if $(a < x)$:

underflow (Number is so

if $(a > y)$:

overflow

little we can't
represent it in
our format)

Question - 12:

Multiply the given numbers using 18 bit IEEE-754 floating-point representation where the size of the fraction field is 12 bits. Check if the result has overflow or underflow.

Note: Consider 10 decimal digits while converting from decimal to binary for the following questions.

	Given Numbers	Result	Overflow/ Underflow
i.	7.01_{10} and 0.71_{10}		
ii.	$0.000101_2 \times 2^{-85}$ and $10.1_2 \times 2^{-90}$		
iii.	$0.0101_2 \times 2^{79}$ and $10010.0101_2 \times 2^{68}$		

ii) fraction $\rightarrow$ 12-bit,
sign $\rightarrow$ 1-bit

biased exponent $\rightarrow 18 - 12 - 1 = 5\text{-bit}$

$$0.000101 \times 2^{-85} \quad \times$$

$$10.1 \times 2^{-90}$$

Normalize,

$$1.01 \times 2^{-85-4} \rightarrow 1.01 \times 2^{-89}$$

$$1.01 \times 2^{-90+1} \rightarrow 1.01 \times 2^{-89}$$

Adding exponents,

$$(1.01 \times 1.01) \times 2^{-89-89}$$

$$= (1.01 \times 1.01) \times 2^{-178}$$

$$= 1.1001 \times 2^{-178}$$

Already in normalized form
so actual exponent is -178

$$\text{Bias} \rightarrow 2^{n-1} - 1 = 2^{5-1} - 1$$

$$= 15$$

$$\text{biased exponent} \rightarrow 15 - 178$$

$$= -163$$

biased exponent range,

$$0 \text{ to } 2^5 - 1$$

$$= 0 \text{ to } 31$$

$$= 1 \text{ to } 30 \text{ (since first and last are reserved)}$$

Now,

$-163 < 1$ so, there is an underflow